THES Site Manpower System — Full Build Instruction for AI Agent

Context & Background

You are building a **construction site manpower management system** for THES Pte Ltd, a construction company operating two sites in Singapore: **TMJP** and **Brani**.

The system replaces 4 separate WhatsApp group chats currently used for:

1. Summary headcount (for management)
2. Detailed headcount with worker IDs (e.g. HB-130, LHL-242)
3. Dormitory headcount (Jalan Papan and Tuas dorms)
4. Overtime approvals and tracking

The existing working codebase is a **Streamlit app** (`main.py` + `db_utils.py`) backed by **SQLite**. It already handles:

- Daily manpower logging per supervisor (name + worker count + remarks)
- Overtime logging per supervisor
- Pending/logged status table
- Excel export
- WhatsApp message generation (summary and detail formats)

Your job is to build **Phase 2 (one source of truth)** and **Phase 3 (full system)** on top of this foundation without breaking what exists.

Existing Database Schema (do not modify these tables, only ADD to them)

```sql
```

```sql
-- Supervisor roster
CREATE TABLE deployments (
    Supervisor       TEXT PRIMARY KEY,
    Site_Location    TEXT,           -- 'TMJP' or 'Brani'
    Planned_Workers  INTEGER DEFAULT 0
);

-- Daily manpower log (one row per supervisor per day)
CREATE TABLE daily_logs (
    date         TEXT,
    supervisor   TEXT,
    workers      INTEGER,
    remarks      TEXT DEFAULT '',
    UNIQUE(date, supervisor)
);

-- Overtime log (one row per supervisor per day)
CREATE TABLE overtime_logs (
    date       TEXT,
    supervisor TEXT,
    ot_workers INTEGER,
    remarks    TEXT DEFAULT '',
    UNIQUE(date, supervisor)
);
```

Existing File Structure

```
project/
├── main.py          # Streamlit UI — tabs: Daily Manpower, Overtime
├── db_utils.py      # All database functions
└── site_management.db
```

Phase 2 — One Source of Truth

Goal

Replace the "detail headcount" and "dormitory" WhatsApp groups with app tabs. Every piece of data lives in one database.

New tables to add in `db_utils.py` → `init_dummy_db()`

```sql
```

```sql
-- Individual worker roster
CREATE TABLE IF NOT EXISTS workers (
    worker_id    TEXT PRIMARY KEY,   -- e.g. 'HB-130', 'LHL-242'
    name         TEXT,
    company      TEXT,               -- 'HB' (Hup Boon) or 'LHL' (LHL company)
    supervisor   TEXT,               -- FK → deployments.Supervisor
    dorm         TEXT,               -- 'Jalan Papan' or 'Tuas'
    status       TEXT DEFAULT 'active'  -- 'active', 'resigned', 'repatriated'
);

-- Daily worker-level attendance (optional detail layer)
CREATE TABLE IF NOT EXISTS worker_attendance (
    date       TEXT,
    worker_id  TEXT,
    status     TEXT,   -- 'present', 'MC', 'AL', 'HL' (home leave), 'absent'
    remarks    TEXT DEFAULT '',
    UNIQUE(date, worker_id)
);

-- Dormitory headcount log
CREATE TABLE IF NOT EXISTS dorm_logs (
    date        TEXT,
    dorm        TEXT,   -- 'Jalan Papan' or 'Tuas'
    supervisor  TEXT,
    headcount   INTEGER,
    UNIQUE(date, dorm, supervisor)
);

-- OT approval requests
CREATE TABLE IF NOT EXISTS ot_requests (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    date        TEXT,
    supervisor  TEXT,
    workers     INTEGER,
    hours       TEXT,    -- e.g. '08:00-22:00'
    site        TEXT,
    remarks     TEXT DEFAULT '',
    status      TEXT DEFAULT 'pending',  -- 'pending', 'approved', 'rejected'
    approved_by TEXT DEFAULT '',
    approved_at TEXT DEFAULT ''
);
```

New functions to write in `db_utils.py`

Worker Roster CRUD

```python
def add_worker(worker_id, name, company, supervisor, dorm)
    # INSERT OR IGNORE into workers table

def get_workers_by_supervisor(supervisor)
    # Returns DataFrame of all active workers under that supervisor
    # Columns: worker_id, name, company, dorm

def get_all_workers()
    # Full roster DataFrame, joined with deployments for site info
    # Columns: worker_id, name, company, supervisor, site_location, dorm, status

def update_worker_status(worker_id, new_status)
    # UPDATE workers SET status = ? WHERE worker_id = ?

def search_worker(query)
    # SELECT * FROM workers WHERE worker_id LIKE ? OR name LIKE ?
    # Used by PM to find a specific worker quickly
```

Worker Attendance

```python
def log_worker_attendance(log_date, worker_id, attendance_status, remarks='')
    # INSERT OR REPLACE into worker_attendance
    # attendance_status: 'present', 'MC', 'AL', 'HL', 'absent'

def get_attendance_by_date(log_date)
    # Returns full DataFrame joined with workers table
    # Columns: worker_id, name, company, supervisor, status, remarks

def get_workers_on_leave(log_date)
    # Returns only rows where status != 'present'
    # This is what PM currently has to ask in WhatsApp

def get_worker_attendance_history(worker_id, days=30)
    # Returns last N days of attendance for one worker
```

Dormitory

```python
def log_dorm_headcount(log_date, dorm, supervisor, headcount)
    # INSERT OR REPLACE into dorm_logs

def get_dorm_summary(log_date)
    # Returns DataFrame grouped by dorm with total headcount
    # Columns: dorm, total_headcount, supervisor_count

def get_dorm_detail(log_date, dorm)
    # Returns per-supervisor breakdown for one dorm on one date
```

OT Approval Flow

```python
def submit_ot_request(date, supervisor, workers, hours, site, remarks='')
    # INSERT into ot_requests with status='pending'
    # Returns the new request id

def get_pending_ot_requests()
    # SELECT * FROM ot_requests WHERE status = 'pending'
    # Used in PM approval view

def approve_ot_request(request_id, approved_by)
    # UPDATE ot_requests SET status='approved', approved_by=?, approved_at=NOW()
    # Also writes to overtime_logs so it flows into existing OT summary

def reject_ot_request(request_id, approved_by, reason='')
    # UPDATE ot_requests SET status='rejected', remarks=reason

def get_ot_history(days=30)
    # Returns all ot_requests joined with approval info for audit trail
```

New tabs to add in `main.py`

Change:

```python
tab1, tab2 = st.tabs(["📋 Daily Manpower", "⏰ Overtime"])
```

To:

```python
tab1, tab2, tab3, tab4, tab5 = st.tabs([
    "📋 Daily Manpower",
    "⏰ Overtime",
    "🏠 Dormitory",
    "👷 Workers",
    "✅ OT Approval"   # only visible/useful for PM role
])
```

Tab 3 — Dormitory (`tab3`)

- Selectbox: Supervisor
- Selectbox: Dorm (Jalan Papan / Tuas)
- Number input: Headcount
- Button: Log
- Summary table: dorm totals for today
- WhatsApp message block: Jalan Papan: X / Tuas: Y / Total: Z

Tab 4 — Workers (`tab4`)

Two sub-sections using `st.expander`:

Section A — Today's Attendance

- Supervisor selectbox to filter
- For each worker under that supervisor: worker_id + name + status dropdown (present/MC/AL/HL/absent) + remarks
- Bulk "Save All" button
- Below: table of workers on leave today (across all supervisors) — this replaces the PM asking in WhatsApp

Section B — Worker Roster

- Full searchable table (use `st.data_editor` so records can be added inline)
- Add worker form: worker_id, name, company, supervisor, dorm
- Filter by supervisor or company

Tab 5 — OT Approval (`tab5`)

Two views based on a radio button: "Supervisor" vs "Manager"

Supervisor view:

- Form: date, workers count, OT hours (text e.g. "08:00–22:00"), site, remarks
- Submit button → writes to `ot_requests` with status=pending
- Table: "Your past OT requests" with status column

Manager view:

- Password gate: `st.text_input(type='password')` checked against a hardcoded string in a `secrets.toml` or env var — keep it simple for now
- Table of all pending requests with Approve / Reject buttons per row
- When approved → calls `approve_ot_request()` which also writes to `overtime_logs`
- Audit trail table below: last 30 days of approved/rejected requests

---

Phase 3 — Full System

Goal

Give the Project Manager a single dashboard with charts, monthly reports, and payroll-ready exports. No manual compilation needed.

New tables to add

```sql
```

```sql
-- Monthly planned headcount targets (set by PM at start of month)
CREATE TABLE IF NOT EXISTS monthly_plan (
    year_month   TEXT,   -- e.g. '2026-04'
    supervisor   TEXT,
    planned      INTEGER,
    UNIQUE(year_month, supervisor)
);

-- Leave applications (formal record, not just remarks)
CREATE TABLE IF NOT EXISTS leave_applications (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    worker_id   TEXT,
    leave_type  TEXT,   -- 'MC', 'AL', 'HL'
    start_date  TEXT,
    end_date    TEXT,
    approved    INTEGER DEFAULT 0,   -- 0 or 1
    remarks     TEXT DEFAULT ''
);
```

New module: `reports.py`

Create this as a separate file — keeps `db_utils.py` focused on CRUD.

```python
```

```python
# reports.py

def get_monthly_summary(year_month)
    """
    Returns DataFrame with columns:
    supervisor, site, planned, actual_avg_daily, total_ot_workers,
    total_leave_days, variance
    Used for PM's monthly report.
    """

def get_daily_trend(site, days=30)
    """
    Returns DataFrame with columns: date, total_workers
    for one site over last N days.
    Used to draw the trend chart.
    """

def get_ot_hours_by_supervisor(year_month)
    """
    Returns DataFrame: supervisor, total_ot_workers_days
    Aggregated from overtime_logs for the month.
    """

def get_leave_summary(year_month)
    """
    Returns DataFrame: worker_id, name, supervisor, MC_days, AL_days, HL_days
    Built from worker_attendance table.
    Used for payroll export.
    """

def export_monthly_payroll(year_month)
    """
    Returns BytesIO Excel with sheets:
    - Summary (supervisor level)
    - Worker Detail (worker_id level: days present, OT days, leave breakdown)
    - Leave Register (all leave applications)
    Formatted for payroll team — no further processing needed.
    """

def export_pm_report(year_month)
    """
    Returns BytesIO Excel formatted for PM's monthly report:
    - Sheet 1: Planned vs Actual chart data (for Excel to render as bar chart)
```

```
    - Sheet 2: OT summary
    - Sheet 3: Leave summary
    - Sheet 4: Raw daily logs
    """
```

New tab in `main.py`: Manager Dashboard

Add a 6th tab: `"📊 Dashboard"` — intended for PM only, same simple password gate as OT approval.

```
Tab layout:
├── Date range picker (default: current month)
├── Site filter (All / TMJP / Brani)
│
├── Row 1 — KPI cards
│   ├── Avg daily headcount (this month)
│   ├── Total OT worker-days
│   ├── Total leave days
│   └── Planned vs actual variance %
│
├── Row 2 — Charts (use st.line_chart / st.bar_chart)
│   ├── Daily headcount trend (line chart, last 30 days)
│   └── Planned vs actual per supervisor (bar chart, current month)
│
├── Row 3 — Workers on leave today (quick lookup table)
│
└── Row 4 — Export buttons
    ├── [📥 Monthly Payroll Export]   → export_monthly_payroll()
    └── [📥 PM Report Export]         → export_pm_report()
```

WhatsApp message generator upgrade

Add a function `generate_all_whatsapp_messages(log_date)` in `db_utils.py` that returns a dict:

```python
{
    "summary":  "...",   # for boss group — same as current
    "detail":   "...",   # with worker IDs listed under each supervisor
    "dorm":     "...",   # Jalan Papan: X / Tuas: Y
    "ot":       "..."    # OT summary if any approved for today
}
```

Each message is pre-formatted, ready to copy-paste into the correct group chat.

File Structure After Full Build

```
project/
├── main.py              # Streamlit UI — all tabs
├── db_utils.py          # All CRUD functions (init, log, get, export)
├── reports.py           # Analytics and export functions (PM-facing)
├── site_management.db   # SQLite database
├── .streamlit/
│   └── secrets.toml     # MANAGER_PASSWORD = "your_password_here"
└── requirements.txt     # streamlit, pandas, openpyxl, plotly (optional)
```

Coding Constraints & Style Rules

Follow these exactly — they match the existing codebase:

1. **Database**: SQLite only. No Postgres, no cloud DB. File path is `'site_management.db'` in the working directory.
2. **Always close connections**: Every function opens its own connection and closes it before returning. Do not share a connection across functions.
3. **Parameterised queries for writes**: Use `?` placeholders for all INSERT/UPDATE. Only use f-strings for SELECT queries with date strings (existing pattern).
4. **No ORM**: Plain `sqlite3` + `pandas.read_sql`. No SQLAlchemy.
5. **Streamlit state**: Use `st.session_state` for anything that needs to persist across reruns within a session (e.g. which tab is active after a button press, manager login state).
6. **No authentication library**: Manager password is checked with a simple `if password == st.secrets["MANAGER_PASSWORD"]` and stored in `st.session_state["is_manager"] = True`. That is enough for internal use.
7. **Return DataFrames from db functions**: UI code in `main.py` should never write raw SQL. All queries go in `db_utils.py` or `reports.py`.
8. **Excel exports use openpyxl**: Already installed. Use `pd.ExcelWriter(buf, engine='openpyxl')`. Auto-size columns using the existing pattern in `export_daily_excel()`.

9. **WhatsApp messages are plain strings**: No HTML, no markdown. Bold uses `*asterisks*` (WhatsApp format). Line breaks use `\n`.
10. **Streamlit version**: Assume latest stable. Use `st.data_editor` for editable tables, `st.columns` for layouts, `st.metric` for KPIs.

Build Order (do this in sequence)

1. Add all new tables to `init_dummy_db()` in `db_utils.py`
2. Add worker CRUD functions to `db_utils.py`
3. Add attendance functions to `db_utils.py`
4. Add dorm functions to `db_utils.py`
5. Add OT approval functions to `db_utils.py`
6. Add Dormitory tab (tab3) to `main.py` — test it works
7. Add Workers tab (tab4) to `main.py` — test both sections
8. Add OT Approval tab (tab5) to `main.py` — test supervisor and manager views
9. Create `reports.py` with all report functions
10. Add Dashboard tab (tab6) to `main.py`
11. Upgrade `generate_all_whatsapp_messages()` to include dorm and OT
12. Test full export: monthly payroll Excel + PM report Excel

---

Sample Data to Seed for Testing

```python
```

```
# Workers — add these in init_dummy_db() using INSERT OR IGNORE
workers = [
    ('HB-130',  'Ahmad Bin Salleh',    'HB',  'Suman_Structure',    'Jalan Papan'),
    ('HB-176',  'Raju Krishnan',        'HB',  'Alim_Lifting',       'Jalan Papan'),
    ('HB-117',  'Mohammed Farhad',      'HB',  'Junayed_Scaffolding','Jalan Papan'),
    ('HB-127',  'Suresh Kumar',         'HB',  'Suman_Structure',    'Tuas'),
    ('LHL-242', 'Farhad Rahman',        'LHL', 'Alim_Lifting',       'Tuas'),
    ('LHL-568', 'Karim Abdullah',       'LHL', 'Junayed_Scaffolding','Jalan Papan'),
    ('LHL-914', 'Milon Hossain',        'LHL', 'Alim_Lifting',       'Tuas'),
    ('LHL-915', 'Sohel Rana',           'LHL', 'Suman_Structure',    'Jalan Papan'),
    ('LHL-923', 'Rubel Islam',          'LHL', 'Suman_Structure',    'Jalan Papan'),
    ('LHL-930', 'Jahangir Alam',        'LHL', 'Suman_Structure',    'Jalan Papan'),
    ('LHL-877', 'Shahin Miah',          'LHL', 'Kanaja_MEP',         'Jalan Papan'),
    ('LHL-398', 'Liton Ahmed',          'LHL', 'Alim_Lifting',       'Tuas'),
    ('LHL-665', 'Rasel Hossain',        'LHL', 'Alim_Lifting',       'Tuas'),
    ('LHL-988', 'Sabbir Rahman',        'LHL', 'Junayed_Scaffolding','Tuas'),
    ('LHL-982', 'Nazmul Haque',         'LHL', 'Junayed_Scaffolding','Tuas'),
]
```

Key Business Logic Notes

- A worker's **company** is encoded in their ID prefix: `HB-` = Hup Boon, `LHL-` = LHL. Parse it from `worker_id.split('-')[0]`.
- **OT approval flow**: Supervisor submits → PM approves in Tab 5 → approval auto-writes to `overtime_logs` → OT tab already picks it up. No double entry.
- **Dormitory headcount** is independent of daily manpower. A worker can be present on site but sleeping in either dorm. Log them separately.
- **Leave in remarks vs attendance table**: Phase 2 introduces the `worker_attendance` table. The existing `remarks` field in `daily_logs` stays as a free-text note for the supervisor. The attendance table is the structured record for reporting.
- **Monthly plan**: Set once per month per supervisor in the `monthly_plan` table. The `Planned_Workers` in `deployments` is the default daily target; `monthly_plan` allows it to vary by month.
- **`worker_attendance` is optional per day** — supervisors who don't fill it still have their headcount logged via `daily_logs`. The detailed attendance is bonus data, not a blocker.

Definition of Done

The system is complete when:

- [ ] A supervisor can open the app, log their headcount + OT request in under 60 seconds
- [ ] A PM can open Tab 5 and approve all pending OT requests in one view
- [ ] A PM can answer "who is on MC today?" in under 10 seconds using Tab 4
- [ ] A PM can answer "what is our headcount vs plan this month?" from the Dashboard tab
- [ ] End of day: one click generates the Excel for payroll
- [ ] End of day: copy-paste WhatsApp messages for all 4 groups are ready in one place
- [ ] Zero WhatsApp groups are needed for operational data entry